It's important to note here that any template containing an {% extends %} tag – which should be the first tag in the template for the sake of sanity – is only permitted to contain content that is wrapped in {% block %} tags (otherwise Twig won't know what to do with it and duly throw an exception). Now, to render the entire thing, we actually tell Twig to render the child…

```
// index.php
print $twig->render('child.html.twig');
```

… because the {% extends %} tag contains the information Twig requires to fetch the parent. The resulting output should be:

```
$ curl -v http://localhost:8080/index.php
<!DOCTYPE html>
<html>
  <head>
    <title>CHILD</title>
  </head>
  <body>
    <div>CHILD CONTENT HERE...</div>
  </body>
</html>
```

You'll have hopefully noticed that not every {% block %} tag in `parent.html.twig` contained content. This is perfectly fine; if the child doesn't provide an override, then the result is just an empty string at that position. It is also possible to access the parent block content from the child via the `parent()` function, which will output the entire parent block content at the position it is called.

Using Other Templates

Twig's inheritance model, like PHP, is single. A child can extend exactly one parent. However, the block mechanism is pretty useful, and you can make use of it via the {% use %} tag, which is sort of like (if you squint a little) multiple inheritance (more accurately, it is horizontal reuse). {% use %} with block tags is a little like {% import %} with macros: the blocks defined in the "used" template become part of the "using" template. The following example should clarify: